

Neptune Bank

Online Banking System

Technical Documentation

Project:	Neptune Bank – Online Banking System
Author:	Ayshi Shannidhya Panda
Technology:	Spring Boot 3.5 Microservices
Language:	Java 21
Database:	PostgreSQL
Messaging:	RabbitMQ
Repository:	https://github.com/ayshishannidhya/Banking_System
Date:	June 2026

Secure Online Banking System with Account Management,
Transaction History, and Fund Transfers

© 2025–2026 Ayshi Shannidhya Panda. All rights reserved.
This document and the associated source code are proprietary and confidential.
Unauthorized reproduction, distribution, or use is strictly prohibited.

Contents

1	Introduction	2
1.1	Project Overview	2
1.2	Key Features	2
1.3	Technology Stack	3
2	System Architecture	4
2.1	Microservices Overview	4
2.2	Inter-Service Communication	4
2.2.1	RabbitMQ Queue Architecture	5
2.3	Security Architecture	5
2.3.1	Authentication Flow	5
2.3.2	JWT Token Structure	5
2.3.3	Password Security	5
2.3.4	Role-Based Access Control	6
3	Service Specifications	7
3.1	Auth Service (Port 8086)	7
3.1.1	Purpose	7
3.1.2	Data Model	7
3.1.3	API Endpoints	7
3.2	User Service (Port 8080)	8
3.2.1	Purpose	8
3.2.2	Data Model	9
3.2.3	API Endpoints	10
3.2.4	RabbitMQ Listener	11
3.3	Account Service (Port 8083)	11
3.3.1	Purpose	11
3.3.2	Data Model	11
3.3.3	Account Number Generation	12
3.3.4	API Endpoints	12
3.3.5	RabbitMQ Listeners	12
3.3.6	Update Authorization	12
3.4	Transaction Service (Port 8084)	13
3.4.1	Purpose	13
3.4.2	Data Model	13
3.4.3	Transaction Status Lifecycle	13
3.4.4	API Endpoints	14
3.4.5	Fund Transfer Flow	14

3.4.6	Transaction History	15
3.5	OTP Service (Port 8082)	15
3.5.1	Purpose	15
3.5.2	Data Model	15
3.5.3	OTP Generation	15
3.5.4	API Endpoints	16
3.5.5	Delivery Channels	16
4	Database Configuration	17
4.1	PostgreSQL Setup	17
4.2	Entity Relationships	17
4.2.1	User Service Entities	17
4.2.2	Account Service Entities	17
4.2.3	Transaction Service Entities	18
5	Error Handling	19
5.1	Global Exception Handling	19
5.2	Error Response Format	19
6	RabbitMQ Configuration	20
6.1	Connection Settings	20
6.2	Message Serialization	20
6.3	RPC Pattern	20
7	Deployment	21
7.1	Prerequisites	21
7.2	Build Commands	21
7.3	Run Order	21
8	API Quick Reference	22
9	License & Intellectual Property	23
9.1	Copyright Notice	23
9.2	Proprietary License Terms	23
9.2.1	Grant of License	23
9.2.2	Restrictions	23
9.2.3	Intellectual Property Rights	24
9.2.4	Confidentiality	24
9.2.5	Termination	24
9.2.6	Attribution Requirement	24
9.2.7	Governing Law	25
9.3	Disclaimer of Warranty	25
9.4	Contact Information	25
10	Conclusion	26

Chapter 1

Introduction

1.1 Project Overview

Neptune Bank is a secure online banking system built using a microservices architecture with Spring Boot 3.5.5 and Java 21. The system provides comprehensive banking functionality including user registration with KYC verification, account management, real-time fund transfers, transaction history tracking, and OTP-based multi-channel authentication.

The system is designed with security as a first-class concern, employing RSA-based JWT authentication, BCrypt password hashing, and role-based access control across all services.

1.2 Key Features

- **User Management:** Complete user registration with KYC document upload (Aadhaar, PAN, photo, signature), profile management with CRUD operations, and soft-delete capability.
- **Account Management:** Multi-type account creation (Savings, Current, Fixed Deposit), branch-based account association, balance management with optimistic locking.
- **Fund Transfers:** Secure inter-account fund transfers with RabbitMQ-based asynchronous debit/credit operations, automatic rollback on failure, and comprehensive transaction tracking.
- **Transaction History:** Complete transaction audit trail with filtering by account, chronological ordering, and detailed transaction metadata.
- **OTP Authentication:** Multi-channel OTP delivery via SMS (TextBee API) and Email (SMTP), with expiry management and verification tracking.
- **Security:** RSA-256 JWT tokens, BCrypt password hashing, role-based authorization (ADMIN, USER, EMPLOYEE), and Spring Security filter chains.

1.3 Technology Stack

Table 1.1: Technology Stack

Component	Technology
Language	Java 21 (LTS)
Framework	Spring Boot 3.5.5
Build Tool	Gradle (Groovy DSL)
Database	PostgreSQL 16
ORM	Spring Data JPA / Hibernate
Messaging	RabbitMQ with Spring AMQP
Authentication	JWT (RS256) with Spring Security
Password Hashing	BCrypt (Spring Security Crypto)
API Documentation	RESTful JSON APIs
Service Discovery	Spring Cloud Netflix Eureka (optional)
SMS Gateway	TextBee API
Email	Spring Mail (Gmail SMTP)

Chapter 2

System Architecture

2.1 Microservices Overview

The Neptune Bank system consists of five independent microservices, each with its own database, running on separate ports.

Table 2.1: Microservices Port Configuration

Service	Port	Database	Responsibility
user-service	8080	UserDB	User registration, profile CRUD, KYC
otp-service	8082	OtpDB	OTP generation, delivery, verification
account-service	8083	AccountDB	Accounts, branches, balances
transaction-service	8084	TransactionDB	Transactions, fund transfers
auth-service	8086	AuthDB	Authentication, JWT, authorization

2.2 Inter-Service Communication

Services communicate asynchronously using RabbitMQ for operations that require cross-service data validation and state changes.

Communication Patterns

- **account-service** → **user-service**: User validation during account creation via RabbitMQ RPC (request/reply pattern).
- **transaction-service** → **account-service**: Account validation, debit, and credit operations during fund transfers via three dedicated RabbitMQ queues.

2.2.1 RabbitMQ Queue Architecture

Table 2.2: RabbitMQ Queues

Queue	Producer	Consumer	Purpose
user-validation-queue	account-service	user-service	Validate user exists
account-validation-queue	transaction-service	account-service	Validate account exists
account-debit-queue	transaction-service	account-service	Debit source account
account-credit-queue	transaction-service	account-service	Credit destination account

2.3 Security Architecture

2.3.1 Authentication Flow

1. Client sends credentials to `POST /auth/login` on auth-service.
2. Auth-service validates credentials using BCrypt comparison.
3. On success, auth-service generates an RS256 JWT token signed with the private key.
4. Client includes the JWT in the `Authorization: Bearer <token>` header for subsequent requests.
5. Each service validates the token using the corresponding RSA public key.

2.3.2 JWT Token Structure

```
{
  "sub": "username",
  "role": "ADMIN | USER | EMPLOYEE",
  "iat": 1718150400,
  "exp": 1718161200
}
```

Listing 2.1: JWT Claims

The token is signed with RS256 (RSA + SHA-256) using a private key (`private.pem`) and validated with the corresponding public key (`public.pem`). Token expiry is set to 3 hours.

2.3.3 Password Security

All user passwords are hashed using `BCryptPasswordEncoder` before storage. The BCrypt algorithm uses a cost factor of 10 (default), generating unique salts for each password. Plaintext passwords are never stored or logged.

2.3.4 Role-Based Access Control

Table 2.3: User Roles

Role	Permissions
ADMIN	Full access: create/modify accounts, change branches, adjust balances, manage users
USER	Limited access: view own accounts, initiate transfers, view transaction history
EMPLOYEE	Intermediate access: verify KYC documents, manage customer accounts

Chapter 3

Service Specifications

3.1 Auth Service (Port 8086)

3.1.1 Purpose

Handles user authentication, JWT token generation and validation, and user credential management with BCrypt password hashing.

3.1.2 Data Model

Table 3.1: Users Entity (auth-service)

Field	Type	Description
id	Long	Auto-generated primary key
username	String	Unique login identifier
password	String	BCrypt-hashed password
role	Role (Enum)	ADMIN, USER, or EMPLOYEE
isAccountNonExpired	Boolean	Account expiry flag (default: true)
isAccountNonLocked	Boolean	Account lock flag (default: true)
isCredentialsNonExpired	Boolean	Credential expiry flag (default: true)
isEnabled	Boolean	Account enabled flag (default: true)

3.1.3 API Endpoints

Table 3.2: Auth Service Endpoints

Method	Path	Description	Auth
POST	/auth/login	Authenticate and get JWT token	No
POST	/auth/add-user	Register new auth credentials	No
GET	/auth/validate	Validate JWT token	No

POST /auth/login

```
{
  "username": "john_doe",
  "password": "securePassword123",
  "mode": "jwt"
}
```

Listing 3.1: Login Request

```
{
  "message": "Login successful",
  "username": "john_doe",
  "role": "USER",
  "token": "eyJhbGciOiJSUzI1NiJ9...",
  "status": "success",
  "code": "200"
}
```

Listing 3.2: Login Response (Success)

The mode field supports two values:

- **jwt**: Returns a JWT token for stateless authentication.
- **session**: Creates a server-side HTTP session (30-minute timeout).

POST /auth/add-user

```
{
  "username": "john_doe",
  "password": "securePassword123",
  "role": "USER"
}
```

Listing 3.3: Register Request

The password is automatically hashed with BCrypt before storage. Duplicate usernames return a 409 Conflict response.

3.2 User Service (Port 8080)

3.2.1 Purpose

Manages user profiles, contact information, nominee details, and KYC document storage with full CRUD capabilities.

3.2.2 Data Model

Table 3.3: Users Entity (user-service)

Field	Type	Description
userid	Long	Auto-generated primary key
firstName	String	User's first name
middleName	String	User's middle name
lastName	String	User's last name
dateOfBirth	LocalDate	Date of birth
gender	Genders (Enum)	MALE, FEMALE, OTHER
fatherName	String	Father's name
motherName	String	Mother's name
maritalStatus	MaritalStatus	MARRIED, UNMARRIED, DIVORCED, WIDOWED
spouseName	String	Spouse name (required if married)
occupation	Occupation (Enum)	Employment type
salary	String	Monthly salary
citizen	String	Citizenship country
category	Category (Enum)	GENERAL, OBC, SC, ST
religion	Religion (Enum)	Religious affiliation
contactDetails	ContactDetails	One-to-one relationship
nominee	NomineeDetails	One-to-one relationship
kycId	Kyc	One-to-one KYC documents
accountId	Long	Linked account (set via messaging)
isActive	Boolean	Active status flag
isBlocked	Boolean	Block status flag
isDeleted	Boolean	Soft-delete flag
createdAt	LocalDateTime	Creation timestamp
updatedAt	LocalDateTime	Last update timestamp

ContactDetails Entity

Stores phone numbers, email addresses, physical addresses (communication and permanent), city, state, zip, landmark, and country.

NomineeDetails Entity

Stores nominee name, relationship, date of birth, mobile number, email, Aadhaar number, PAN number, and address.

KYC Entity

Stores identity document numbers and scanned images (as `BYTEA` in PostgreSQL):

- Aadhaar card (number + image, mandatory)
- PAN card (number + image, mandatory)

- User photo (mandatory)
- User signature (mandatory)
- Voter ID (optional)
- Passport (optional)
- Driving License (optional)

Each document has a verification status flag and can be verified by an employee.

3.2.3 API Endpoints

Table 3.4: User Service Endpoints

Method	Path	Description	Auth
POST	/auth/user	Register user (multipart)	No
GET	/auth/user/{id}	Get user by ID	Yes
GET	/auth/users	Get all active users	Yes
PUT	/auth/user/{id}	Update user details	Yes
DELETE	/auth/user/{id}	Soft-delete user	Yes
GET	/auth/user/exists/{id}	Check user existence	Yes

POST /auth/user (Registration)

This endpoint accepts `multipart/form-data` with the following parts:

Table 3.5: Registration Multipart Parts

Part Name	Type	Required	Description
user	JSON	Yes	UsersRequestDTO with all user details
aadhaar	File	Yes	Scanned Aadhaar card image
pan	File	Yes	Scanned PAN card image
photo	File	Yes	User photograph
signature	File	Yes	User signature scan
voter	File	No	Voter ID image
passport	File	No	Passport image
driving	File	No	Driving license image

PUT /auth/user/{id} (Partial Update)

Supports partial updates — only non-null fields in the request body are updated. Contact details can also be partially updated within the same request.

DELETE /auth/user/{id} (Soft Delete)

Sets `isDeleted = true` and `isActive = false`. The user record is retained in the database for audit purposes but excluded from `GET /auth/users` results.

3.2.4 RabbitMQ Listener

```

1 @RabbitListener(queues = "${rabbitmq.user.validation.request.queue}")
2 public Boolean validateUser(Long userId) {
3     if (userId == null) return false;
4     return userRepository.existsById(userId);
5 }

```

Listing 3.4: User Validation Listener

This listener responds to RPC requests from account-service to validate whether a user exists before account creation.

3.3 Account Service (Port 8083)

3.3.1 Purpose

Manages bank accounts, branches, and balances. Provides RabbitMQ listeners for account validation and balance operations consumed by transaction-service.

3.3.2 Data Model

Table 3.6: Account Entity

Field	Type	Description
accountId	Long	Auto-generated primary key
userId	Long	Owner's user ID
accountNumber	String	Unique account number (e.g., NB4A7F2E3C1D8B)
branch	Branch	Many-to-one branch association
balance	BigDecimal	Current balance (default: 0.00)
accountType	AccountType	SAVINGS, CURRENT, FIXED_DEPOSIT
modeOfOperation	ModeOfOperation	SINGLE, JOINT, EITHER_OR_SURVIVOR
version	Long	Optimistic locking version
createdAt	LocalDateTime	Account creation timestamp
updatedAt	LocalDateTime	Last modification timestamp

Table 3.7: Branch Entity

Field	Type	Description
branchId	Long	Auto-generated primary key
branchCode	String	Unique branch identifier
branchName	String	Branch display name
branchAddress	String	Physical address
branchCity	String	City
branchState	String	State
branchZip	String	ZIP/PIN code

3.3.3 Account Number Generation

Account numbers are generated as NB followed by 12 random uppercase alphanumeric characters derived from UUID. Uniqueness is guaranteed by checking against existing records with up to 20 retry attempts.

3.3.4 API Endpoints

Table 3.8: Account Service Endpoints

Method	Path	Description	Auth
POST	/api/accounts/create	Create new account	Yes
GET	/api/accounts/get/{id}	Get account by ID	Yes
GET	/api/accounts/get/all	Get all accounts	Yes
GET	/api/accounts/user/{userId}	Get accounts by user	Yes
GET	/api/accounts/number/{num}	Get account by number	Yes
PUT	/api/accounts/update/{id}	Update account	Yes
DELETE	/api/accounts/delete/{id}	Delete account	Yes

3.3.5 RabbitMQ Listeners

The account-service exposes three RabbitMQ listeners for inter-service operations:

1. **Account Validation Listener** (`account-validation-queue`): Receives an account number string, returns `true` if the account exists.
2. **Debit Listener** (`account-debit-queue`): Receives `{accountNumber, amount}`, debits the balance atomically. Returns `false` if insufficient funds.
3. **Credit Listener** (`account-credit-queue`): Receives `{accountNumber, amount}`, credits the balance atomically.

3.3.6 Update Authorization

Account updates are subject to role-based restrictions:

- **Account Owner**: Can only change the mode of operation.
- **Admin**: Can change account type, branch, balance, and ownership (`userId`).

3.4 Transaction Service (Port 8084)

3.4.1 Purpose

Records all financial transactions and orchestrates fund transfers between accounts via RabbitMQ-based communication with account-service.

3.4.2 Data Model

Table 3.9: Transaction Entity

Field	Type	Description
id	Long	Auto-generated primary key
transactionId	String	Unique ID (e.g., NEFT followed by 60 digits)
sourceAccount	Long	Source account identifier
destinationAccount	Long	Destination account identifier
transactionType	TransactionType	DEBIT, CREDIT, TRANSFER, REFUND, etc.
modeOfTransaction	ModeOfTransaction	ONLINE, ATM, UPI, MOBILE_BANKING, etc.
transactionMedium	TransactionMedium	NEFT, RTGS, IMPS, UPI, OTHER
amount	BigInteger	Transfer amount
sourceOrDestinationBankId	SourceOrDestinationBank	Bank details (ManyToOne)
description	String	Transaction description
remarks	String	Additional remarks
transactionDate	LocalDateTime	When the transaction occurred
transactionStatus	TransactionStatus	PENDING, SUCCESSFUL, FAILED, etc.
createdAt	LocalDateTime	Record creation time
updatedAt	LocalDateTime	Last update time

3.4.3 Transaction Status Lifecycle

The system supports 30+ transaction statuses. The primary lifecycle for fund transfers is:

PENDING → SUCCESSFUL
 PENDING → FAILED_INSUFFICIENT_FUNDS
 PENDING → FAILED_TECHNICAL_ERROR

3.4.4 API Endpoints

Table 3.10: Transaction Service Endpoints

Method	Path	Description	Auth
POST	/transactions/create	Create a transaction record	Yes
POST	/transactions/transfer	Execute fund transfer	Yes
GET	/transactions/{txnId}	Get transaction by ID	Yes
GET	/transactions/account/{accId}	Transaction history by account	Yes
GET	/transactions/all	Get all transactions	Yes

POST /transactions/transfer (Fund Transfer)

```
{
  "sourceAccountNumber": "NB4A7F2E3C1D8B",
  "destinationAccountNumber": "NB9X2K5M7P3Q1R",
  "amount": 50000,
  "description": "Rent payment",
  "remarks": "June 2026",
  "sourceOrDestinationBank": {
    "bankName": "Neptune Bank",
    "bankIFSCCode": "NPTB0001234",
    "bankBranchName": "Main Branch",
    "bankBranchCity": "Mumbai",
    "bankBranchState": "Maharashtra",
    "bankBranchCountry": "India",
    "bankBranchZipCode": "400001"
  }
}
```

Listing 3.5: Fund Transfer Request

```
{
  "status": "success",
  "message": "Fund transfer completed successfully.",
  "transactionId": "NEFT843927156...",
  "amount": "50000",
  "code": "201"
}
```

Listing 3.6: Fund Transfer Response (Success)

3.4.5 Fund Transfer Flow

The fund transfer process follows a saga-like pattern with compensating transactions:

1. **Validation:** Verify source and destination accounts are different and both exist (via RabbitMQ to account-service).
2. **Record:** Save a new transaction record with status `PENDING`.
3. **Debit:** Send debit request to account-service via `account-debit-queue`.

4. **Credit:** On successful debit, send credit request via `account-credit-queue`.
5. **Rollback:** If credit fails, automatically reverse the debit by crediting the source account back.
6. **Finalize:** Update transaction status to `SUCCESSFUL` or appropriate failure status.

Failure Handling

If the debit succeeds but the credit fails, the system automatically reverses the debit by crediting the amount back to the source account, and the transaction is marked as `FAILED_TECHNICAL_ERROR`.

3.4.6 Transaction History

The `GET /transactions/account/{accountId}` endpoint returns all transactions where the given account is either the source or destination, ordered by transaction date descending (most recent first).

3.5 OTP Service (Port 8082)

3.5.1 Purpose

Provides multi-channel OTP generation, delivery (SMS and Email), and verification for transaction authorization and user verification workflows.

3.5.2 Data Model

Table 3.11: OtpRecord Entity

Field	Type	Description
id	Long	Auto-generated primary key
identifier	String	Phone number or email (indexed)
otp	String	6-digit generated OTP (indexed)
createdAt	LocalDateTime	Generation timestamp
expiryTime	LocalDateTime	Expiration (5 minutes from creation)
isUsed	Boolean	Whether the OTP has been consumed
isOtpExpired	Boolean	Whether the OTP has expired

3.5.3 OTP Generation

OTPs are generated using `java.security.SecureRandom` to ensure cryptographic randomness. Each OTP is a 6-digit numeric string.

3.5.4 API Endpoints

Table 3.12: OTP Service Endpoints

Method	Path	Description	Auth
POST	/api/otp/send	Send OTP via SMS and/or Email	No
POST	/api/otp/verify	Verify an OTP	No

POST /api/otp/send

```
{
  "phone": "+919876543210",
  "email": "user@example.com"
}
```

Listing 3.7: OTP Send Request

Both channels are optional but at least one must be provided. The service prevents duplicate active OTPs for the same identifier, returning 429 Too Many Requests if an active OTP already exists.

3.5.5 Delivery Channels

- **SMS:** Delivered via TextBee API gateway using REST calls.
- **Email:** Sent via Gmail SMTP using Spring Mail with `SimpleMailMessage`.

Chapter 4

Database Configuration

4.1 PostgreSQL Setup

All services connect to a PostgreSQL instance on `localhost:5433` with the following databases:

Table 4.1: Database Configuration

Service	Database	Username	DDL Mode
auth-service	AuthDB	neptune	update
user-service	UserDB	neptune	update
otp-service	OtpDB	neptune	create
account-service	AccountDB	neptune	update
transaction-service	TransactionDB	neptune	update

DDL Auto Configuration

Most services use `spring.jpa.hibernate.ddl-auto=update`, which creates tables on first run and adds new columns/tables on subsequent runs without dropping existing data. This ensures data persistence across service restarts.

4.2 Entity Relationships

4.2.1 User Service Entities

- `Users 1 : 1 ContactDetails` (bidirectional, cascade ALL)
- `Users 1 : 1 NomineeDetails` (bidirectional, cascade ALL)
- `Users 1 : 1 Kyc` (bidirectional, cascade ALL)

4.2.2 Account Service Entities

- `Branch 1 : N Account` (one branch has many accounts)

4.2.3 Transaction Service Entities

- `SourceOrDestinationBank` $1:N$ `Transaction` (one bank entry links to many transactions)

Chapter 5

Error Handling

5.1 Global Exception Handling

Each service implements a `@RestControllerAdvice` global exception handler that provides consistent error responses across the system.

Table 5.1: Exception Handling Matrix

Exception	HTTP	Services	Scenario
EntityNotFoundException	404	user, transaction	Resource not found by ID
IllegalArgumentException	400	user	Invalid input (e.g., missing spouse name)
ConstraintViolationException	400	user, transaction, otp	Bean validation failures
MethodArgumentNotValidException	400	All	@Valid DTO validation errors
Exception (generic)	500	All	Unexpected runtime errors

5.2 Error Response Format

```
{
  "success": false,
  "message": "User not found with ID: 42",
  "errors": ["field: validation message"]
}
```

Listing 5.1: Standard Error Response

Chapter 6

RabbitMQ Configuration

6.1 Connection Settings

All services sharing RabbitMQ use the following connection parameters:

```
spring.rabbitmq.host=localhost
spring.rabbitmq.port=5672
spring.rabbitmq.username=neptune
spring.rabbitmq.password=ankit
spring.rabbitmq.listener.simple.concurrency=3
spring.rabbitmq.listener.simple.max-concurrency=10
```

Listing 6.1: RabbitMQ Properties

6.2 Message Serialization

All services use `Jackson2JsonMessageConverter` for JSON-based message serialization/deserialization over RabbitMQ, ensuring type-safe communication between producers and consumers.

6.3 RPC Pattern

The system uses RabbitMQ's request/reply pattern (`convertSendAndReceive`) with a 5-second reply timeout. This provides synchronous-like behavior over asynchronous messaging for operations that require immediate confirmation (e.g., account validation before proceeding with a transfer).

Chapter 7

Deployment

7.1 Prerequisites

1. Java 21 JDK installed
2. PostgreSQL 16 running on port 5433
3. RabbitMQ running on port 5672
4. Required databases created: AuthDB, UserDB, OtpDB, AccountDB, TransactionDB
5. RSA key pair (`private.pem`, `public.pem`) in each service's `src/main/resources/`

7.2 Build Commands

```
cd auth-service && ./gradlew build -x test
cd user-service && ./gradlew build -x test
cd otp-service && ./gradlew build -x test
cd account-service && ./gradlew build -x test
cd transaction-service && ./gradlew build -x test
```

Listing 7.1: Building All Services

7.3 Run Order

Services should be started in the following order to satisfy dependencies:

1. **auth-service** (port 8086) – no dependencies
2. **otp-service** (port 8082) – no inter-service dependencies
3. **user-service** (port 8080) – RabbitMQ listener (consumed by account-service)
4. **account-service** (port 8083) – depends on user-service via RabbitMQ
5. **transaction-service** (port 8084) – depends on account-service via RabbitMQ

Chapter 8

API Quick Reference

Table 8.1: Complete API Endpoint Reference

Method	Path	Service	Description
POST	/auth/login	auth	Login
POST	/auth/add-user	auth	Register credentials
GET	/auth/validate	auth	Validate JWT
POST	/auth/user	user	Register user
GET	/auth/user/{id}	user	Get user
GET	/auth/users	user	List all users
PUT	/auth/user/{id}	user	Update user
DELETE	/auth/user/{id}	user	Delete user
GET	/auth/user/exists/{id}	user	Check existence
POST	/api/accounts/create	account	Create account
GET	/api/accounts/get/{id}	account	Get by ID
GET	/api/accounts/get/all	account	List all
GET	/api/accounts/user/{uid}	account	By user ID
GET	/api/accounts/number/{num}	account	By number
PUT	/api/accounts/update/{id}	account	Update
DELETE	/api/accounts/delete/{id}	account	Delete
POST	/transactions/create	transaction	Create record
POST	/transactions/transfer	transaction	Fund transfer
GET	/transactions/{txnId}	transaction	Get by ID
GET	/transactions/account/{id}	transaction	History
GET	/transactions/all	transaction	List all
POST	/api/otp/send	otp	Send OTP
POST	/api/otp/verify	otp	Verify OTP

Chapter 9

License & Intellectual Property

9.1 Copyright Notice

© 2025–2026 Ayshi Shannidhya Panda

All rights reserved.

This software, including all source code, documentation, configuration files, build scripts, and associated materials (collectively, the “Software”), is the original intellectual property of **Ayshi Shannidhya Panda**. The Software was independently designed, developed, and tested by the Author.

9.2 Proprietary License Terms

The Software is distributed under a **Proprietary Software License Agreement**. The full license text is included in the LICENSE file at the root of the project repository.

9.2.1 Grant of License

Subject to the terms of the license agreement, the Author grants the Licensee a **non-exclusive, non-transferable, revocable** license to:

- View and review the Software for evaluation and reference purposes.
- Use the Software internally within the Licensee’s organization solely for the purposes agreed upon between the Author and the Licensee.

9.2.2 Restrictions

The Licensee **SHALL NOT**, without the prior written consent of the Author:

1. **Copy or Reproduce** the Software or any portion thereof, except for reasonable backup purposes.
2. **Modify, Adapt, or Create Derivative Works** based on the Software, including reverse engineering, decompilation, or disassembly.

3. **Distribute, Sublicense, Sell, or Transfer** the Software or any rights therein to any third party.
4. **Remove or Alter** any copyright notices, proprietary legends, or confidentiality markings.
5. **Use Commercially** the Software for resale, SaaS offerings, or integration into commercial products without a separate commercial license.
6. **Claim Authorship or Ownership** of the Software or any portion thereof.
7. **Use Competitively** the Software against the Author or the Author's interests.

9.2.3 Intellectual Property Rights

All intellectual property rights, including but not limited to copyrights, patents, trade secrets, and trademarks, in and to the Software **remain the exclusive property of the Author**. The license agreement does not transfer any ownership rights to the Licensee.

9.2.4 Confidentiality

The Licensee acknowledges that the Software contains confidential and proprietary information. The Licensee agrees to:

- Maintain the confidentiality of the Software.
- Not disclose the Software to any third party without the Author's prior written consent.
- Take all reasonable measures to protect the Software from unauthorized access, use, or disclosure.

9.2.5 Termination

- The license is effective until terminated by the Author.
- The Author may terminate the license at any time by providing written notice.
- The license automatically terminates upon any breach of terms.
- Upon termination, the Licensee must cease all use and destroy all copies.

9.2.6 Attribution Requirement

Any permitted use of the Software must include the following attribution:

Neptune Bank - Online Banking System
Copyright (c) 2025-2026 Ayshi Shannidhya Panda. All rights reserved.

9.2.7 Governing Law

This agreement is governed by and construed in accordance with the laws of **India**. Any disputes arising under this agreement shall be subject to the exclusive jurisdiction of the courts located in India.

9.3 Disclaimer of Warranty

No Warranty

THE SOFTWARE IS PROVIDED “AS IS” WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NONINFRINGEMENT. THE AUTHOR SHALL NOT BE LIABLE FOR ANY CLAIM, DAMAGES, OR OTHER LIABILITY ARISING FROM THE USE OF THE SOFTWARE.

9.4 Contact Information

For licensing inquiries, commercial use requests, or permissions:

Name: Ayshi Shannidhya Panda
Email: a.shannidhya@gmail.com
GitHub: <https://github.com/ayshishannidhya>

Chapter 10

Conclusion

The Neptune Bank Online Banking System provides a comprehensive, secure, and scalable banking platform built on modern microservices principles. The system successfully implements:

- **Security:** RSA-based JWT authentication with BCrypt password hashing
- **Account Management:** Full lifecycle management with branch associations and role-based access control
- **Fund Transfers:** Reliable inter-account transfers with saga-pattern compensation
- **Transaction History:** Complete audit trail with chronological tracking
- **User Management:** Comprehensive user profiles with KYC document management
- **OTP Verification:** Multi-channel (SMS + Email) one-time password system

The microservices architecture ensures independent scalability, fault isolation, and technology flexibility for each service component. The use of RabbitMQ for inter-service communication provides reliable asynchronous messaging with request/reply patterns for synchronous-like operations.